Hyperledger Fabric Framework for Residue-Safe Cardamom Supply Chains and Smart Payments

ANONYMOUS AUTHOR, Anonymous Institution, Anonymous Country

The cardamom supply chain is plagued by the lack of transparency and indiscriminate use of pesticides, which results in produce containing pesticide residues far above the permissible levels. Furthermore, in traditional auction systems, which are opaque and lack price transparency, farmers often receive unfair or below-market prices for their produce. The present study proposes a blockchain-based system that uses Hyperledger Fabric to record the complete journey of cardamom from farmers to end consumers along with facilitating a decentralized auction system to sell and purchase the produce. To reduce the storage pressure of the blockchain, the Interplanetary File System (IPFS) is also integrated into it. The system supports role-based access control, ensures immutable records, and offers packet-code-driven real-time tracking, smart contract-driven payments, competitive price discovery, and incentive-driven market access for farmers. The proposed system enhances traceability and transparency in financial transactions and builds trust among stakeholders by providing a tamper-proof shared ledger where financing terms and payments are visible to all parties, ensuring fairness, reducing disputes, and allowing each participant to verify transactions independently. The system was functionally validated and tested using a benchmarking tool called Hyperledger Caliper under varying transaction loads, with smart contracts specifically written to avoid multiple version concurrency errors—a major bottleneck in Hyperledger Fabric. This approach reduced transaction conflicts and improved throughput, confirming the system's feasibility, scalability, and potential to enhance trust, traceability, and fairness in the cardamom trade ecosystem.

CCS Concepts: • **Applied computing** → **Agriculture**; • **Information systems** → **Supply chain management**; • **Security and privacy** → *Distributed systems security*.

Additional Key Words and Phrases: Cardamom supply chain, Blockchain, Hyperledger Fabric, Smart contracts, Traceability, Agri-food finance, InterPlanetary File System (IPFS), Auction system, Hyperledger Caliper

ACM Reference Format:
Anonymous Author. 2026. Hyperledger Fabric Framework for Residue-Safe Cardamom Supply Chains and Smart Payments. In *Proceedings of Submitted manuscript (Submitted Manuscript).* ACM, New York, NY, USA, 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

.

Small cardamom (Elettaria cardamomum (L.) Maton; Zingiberaceae), often referred to as the "Queen of Spices," is a high-value perennial crop widely cultivated in tropical regions. It belongs to the family Zingiberaceae and is valued for its aromatic seeds used in culinary and medicinal applications. India is the second largest producer as well as the largest consumer of cardamom, consuming around 24,463 tonnes of dried cardamom (**?**). Among all the states in India, Kerala is the largest producer, contributing around 90 percent of the total production with 40,345 ha under cultivation, followed by Karnataka and Tamil Nadu (**?**). In the last few decades, the cardamom industry has witnessed a surge in areas under cultivation and a substantial increase in productivity. A study analyzing productivity trends in the Cardamom Hills region of Kerala reported a notable rise in yields from about 130 kg per hectare in 1997 to 400 kg per hectare by 2016, representing a near threefold increase over 20 years (**?**). Novel agricultural practices, modern fertilizers, and pesticides enabled this productivity surge. But this rapid rise in production levels entailed a host of problems, with farmers resorting to indiscriminate application of pesticides to forestall any kind of pest attack.

Author's Contact Information: Anonymous Author, Anonymous Institution, Anonymous City, Anonymous State, Anonymous Country.

The proliferation of insects and pests has forced farmers to rely on pesticides rather than employing sustainable practices. Almost all the plantations apply a cocktail of pesticides every 20-30 days to prevent any kind of pest attacks (**?**). This practice has been condemned by the agricultural scientists who issued warnings about the severe health hazards it entails (**?**). Indian farmers widely use pesticides and herbicides banned by the European Union to control pest attacks. Indian cardamom exports are concentrated in the Middle East, with their share of global exports declining from 28% in 1988 to 9% in 2020 by quantity and from 42.5% to 9% by value. In 2019, India's contribution to Saudi Arabia's imports decreased primarily due to the rejection of shipments that contained higher concentrations of hazardous pesticide residues (**?**). The produce that gets rejected in the global market finds its way into the Indian markets, where food safety often takes a backseat because of limited financial resources, lack of traceability, and slack government interventions (**?**). The end consumers remain ignorant about pesticide content and the health hazards it entails. On 4 January 2023, a food testing laboratory in Trivandrum released a report, finding pesticide residues beyond permissible limits in cardamom used in Aravana payasam, the famous sacred food given to devotees at the Sabarimala Lord Ayyappa Temple (**?**). The Kerala High Court raised concerns about the pesticide residue in cardamom and its implications on public health. Many of the problems largely lie in the lack of transparency in the cardamom supply chain and the end consumers, who largely remain oblivious to the provenance of the food. The lack of transparency also affects the importers, who often get deceived by the fraudulent exporters who mix low-quality cardamom with premium-grade cardamom (**?**).

In addition to safety concerns, the cardamom sector also faces serious financial challenges. Farmers often fall victim to delayed payments or outright fraud in auction centers, where buyers sometimes collect produce without honoring payment commitments. These financial irregularities increase the vulnerability of smallholders, who already operate with limited bargaining power and no real-time visibility in the transaction trail. Most of the sales occur through regional auction centers, where the produce is auctioned by the auction center after charging considerable fees. These auctions occur without any transparency and do not give the farmer a choice to accept the bid price. Auctioneers exploit this loophole by colluding to slash the bid price, resulting in lower prices for farmers. In 2018, the Spices Board flagged a major auctioneer for forged documents and delayed payments (**?**). Sales through local traders are also fraught with risks. In August 2024, Idukki district cardamom farmers filed 32 complaints against a buyer who collected produce from around 1,400 bills but failed to pay, a scam totaling approximately 18 crore Indian rupees (**?**).

Incentivizing farmers to produce cardamom with minimal application of pesticides and empowering consumers to trace the provenance of the food can be an effective strategy to curb the indiscriminate application of pesticides. Although digital technologies like Near Field Communication (NFC), Radio Frequency Identification (RFID), and Quick Response (QR) codes have been widely used in the supply chain to track and trace the product, they are ineffective for fragmented data and lack user-friendliness. Furthermore, most food supply chains rely on centralized data management systems that are vulnerable to single points of failure and data tampering and offer no accountability in the event of product recalls. In addition, traditional payment systems often lack transparency and delay settlements, leaving farmers undercompensated and financially insecure ([**?**]). A decentralized and tamper-proof traceability platform that integrates quality verification, transaction transparency, and payment traceability can address food safety concerns and mitigate financial exploitation, enhancing trust and equity for all stakeholders.

Blockchain is a system that not only enables traceability but also facilitates secure, transparent, and automated financial transactions, effectively addressing many of the aforementioned challenges and uniting various stakeholders on one platform. Although many organizations like the World Wide Fund (WWF) and the Food and Agriculture Organization of the United Nations (FAO) have successfully deployed blockchain-based traceability systems to trace the

produce (?), no such initiatives have been taken to address the challenges facing the cardamom supply chain. Therefore, a blockchain-based supply chain system, which offers unique capabilities of immutability, transparency, and decentralization, has the potential to address the challenges facing the cardamom industry.

This paper proposes a blockchain-based traceability system for the cardamom supply chain implemented using Hyperledger Fabric. The proposed system mainly employs the blockchain technology to trace the produce, bringing together various stakeholders like farmers, consumers, aggregators, retailers, etc., enabling the end consumers to track the produce with the help of a packet code printed on the packets. Beyond traceability, the platform integrates smart contract–driven auctions for competitive price discovery and an incentive mechanism where farmer ratings improve with pesticide-compliant lots, encouraging safer practices. The design of the proposed platform encourages farmers to produce cardamom with minimal pesticide limits. It also offers role-based access to various stakeholders and enables the real-time tracking of the product and facilitates secure, transparent, and automated financial transactions. This research addresses the aforementioned challenges. First, it addresses the traceability challenges in the cardamom supply chain by designing a blockchain-based system architecture that encompasses all stakeholders, including farmers, aggregators, retailers, and consumers. Second, modular smart contracts are written to facilitate transparency and traceability in the supply chain. Third, a web-based interface is created and a Representational State Transfer (REST) Application Programming Interface (API) layer to enable stakeholder interactions and provide end-to-end visibility. In addition to improving traceability, this study integrates a financial transaction layer that facilitates timely payments to farmers from retailers. Separate wallets are created for the stakeholders through which the users can send and receive money. All the payments between the stakeholders are smart contract driven, which enables timely and secure money transfer. Furthermore, the system was rigorously benchmarked using Hyperledger Caliper to evaluate throughput and latency under varying workloads, and MVCC-related concurrency issues were mitigated through optimized append-only data structures and conflict-aware transaction design, enhancing system scalability.

2 Related Work

Agri-food supply chains involve the movement of agricultural products from producers to consumers through several stakeholders, including farmers, processors, distributors, retailers, and consumers [??]. In such systems, food safety, transparency, and traceability are essential [?]. For cardamom, these concerns are especially important because pesticide residues, product authenticity, and payment transparency directly affect farmers, traders, and consumers. Blockchain technology has therefore gained attention as a tool for improving trust, traceability, and accountability in agri-food supply chains [??]. It can also reduce dependence on intermediaries and support more transparent transactions [?].

2.1 Blockchain Framework

Blockchain records transactions in a decentralized and tamper-resistant ledger. Based on access control, blockchain systems are generally classified as permissionless or permissioned. Permissionless platforms such as Ethereum allow open participation, while permissioned frameworks such as Hyperledger Fabric provide controlled access, privacy, and better suitability for enterprise use [??]. Hyperledger Fabric is particularly useful for supply chain applications because it supports modular design, role-based access control, and efficient consensus mechanisms [?].

2.2 Hyperledger-Based Applications in Agricultural Supply Chains

Earlier Hyperledger Fabric studies in agricultural supply chains mainly focus on traceability, food safety, and secure data sharing, as shown in Table 1. These works demonstrate the value of permissioned blockchain for trusted record keeping, but most remain limited to track-and-trace functions. They rarely address practical trade needs such as automated payments, farmer incentives, and fair price discovery. Even IoT-based models, such as [?], are often tested in controlled settings. Thus, a gap remains in developing active blockchain platforms that combine traceability with smart contract-driven financial and incentive mechanisms.

Table 1. Core Hyperledger Fabric-Based Agricultural Supply Chain Systems

Author	Methodology	Key Contribution	Research Gap
[?]	System design and prototype implementation	Proposes a blockchain-based architecture for improving transparency and traceability in agri-food supply chains using permissioned networks	focuses solely on logistics; lacks automated payment triggers (Smart Contracts) to link physical delivery with immediate financial settlement.
[?] XXin Zhang et al.	Implementation with experimental validation	Develops a secure traceability system integrating blockchain with IoT data for monitoring agricultural products	Focuses on historical data logging rather than real-time trade; there is no provision for automated auction mechanisms that allow buyers to bid on products based on live IoT-verified conditions.
[?].	System development and case study	Implements a decentralized system to enhance trust, data integrity, and traceability across stakeholders	Lack of incentive-driven engagement; does not provide a reputation-based or tokenomic reward system to encourage active and honest data participation by stakeholders.
[?] Hu et al.	Prototype implementation with validation	Proposes a blockchain-based food safety system ensuring end-to-end traceability and tamper-proof records	Static financial model; lacks automated payment settlement and financial integration, missing the link between physical product tracking and fiscal automation.

Table 2 shows that recent blockchain systems support features such as dynamic pricing, secure bidding, and traceability. However, they rarely connect produce quality with farmer market access. They also provide limited analysis of concurrent workloads and do not fully address scalability issues such as MVCC conflicts.

The proposed system addresses these gaps by combining decentralized auction, smart-contract-driven payments, and pesticide-compliance-based incentives. It uses append-only updates to reduce MVCC conflicts and evaluates performance through structured benchmarking. This design supports fair price discovery, scalable execution, and stronger incentives for safe farming practices.

Table 3 compares studies that evaluate blockchain performance using Hyperledger Caliper or simulation-based methods. Prior works mainly analyse throughput and latency under different workloads [????], while [?] examine Fabric performance under different block sizes and endorsement policies. However, most studies are tested under moderate loads or controlled settings. They rarely examine high-frequency transaction patterns, such as auctions, where many users may submit bids and payments concurrently. They also give limited attention to MVCC conflicts, which can reduce transaction success under concurrent workloads.

To address these limitations, this study develops a Hyperledger Fabric-based cardamom supply chain system with three main innovations. First, it combines traceability with smart contract-driven payments, allowing cardamom lots

Table 2. Comparison of State-of-the-Art (SOTA) Blockchain Systems (2023–2026)

Author	Methodology	Key Contribution	Research Gap
[?]	STP framework with HLF + IPFS	Integrates dynamic pricing and traceability for seafood supply chains using automated pricing smart contracts	Lacks a decentralized auction mechanism and specialized MVCC conflict resolution for high-frequency bidding
[?]	System design with Caliper benchmarking	Develops an end-to-end halal certification system with IPFS integration and detailed performance evaluation	Focus is limited to tracking/certification; does not address concurrency bottlenecks or trading-driven transaction loads
[?] Yuan Zhang et al. (2023)	HLF + ML integration	Proposes a traceability model using machine learning for outlier detection and lightweight storage via IPFS	Explicitly identifies MVCC and concurrency issues as unresolved bottlenecks for large-scale agricultural deployment
[?]	Security-centric system design	Proposes a dual-mode auction system for general agriculture using zk-SNARKs for enhanced transaction privacy	High computational overhead due to zero-knowledge proofs; lacks quantitative Caliper-based throughput validation
[?]	Pricing transparency and IPFS integration	Implements a Hyperledger Fabric-based rubber supply chain in Cambodia; uses IPFS for large data storage and enhances pricing transparency for smallholders	Focuses primarily on high-level traceability; lacks specific smart-contract optimizations for handling high-concurrency auction bidding
[?]	Anti-counterfeiting and Caliper testing	Proposes a tea traceability scheme using Hyperledger Fabric and IPFS; provides initial Hyperledger Caliper benchmarks for read/write operations	Benchmarking is limited to low transaction rates; does not address MVCC-related failure patterns in competitive auction scenarios

Table 3. Performance Evaluation and Benchmarking of Blockchain-Based Systems

Author	Methodology	Key Contribution	Research Gap
[?]	System design with Caliper benchmarking	Implements a multi-organization Fabric system with load balancing, achieving 117 TPS and 100% success rates	Tested on a limited two-organization setup; lacks large-scale multi-channel validation and high-concurrency conflict analysis (MVCC)
[?] et al. (2025)	Experimental evaluation with Caliper + IPFS	Integrates blockchain, IoT, and IPFS for decentralized records, evaluating throughput and latency metrics	Lacks stress testing under extreme transaction rates and does not address state database contention issues
[?]	Experimental benchmarking using Caliper	Provides a foundational analysis of Hyperledger Fabric throughput and latency bottlenecks	Focuses on generic chaincode; lacks application-specific logic (e.g., auctions) and real-world supply chain workloads
[?]	Experimental evaluation of Hyperledger Fabric	Analyzes performance of Hyperledger Fabric under different block sizes and endorsement policies for agri-food	Does not consider application-level contention or MVCC conflicts arising from concurrent transaction workloads

and packets to be tracked while financial transactions are executed transparently. Second, it introduces an incentive mechanism in which pesticide-compliant lots improve farmer visibility and bidding outcomes. Third, it adopts an MVCC-aware smart contract design using append-only transaction records and composite-key structures to reduce conflicts under concurrent bidding and payment workloads. The system is evaluated using Hyperledger Caliper under high transaction rates with realistic bidding, payment, and asset-transfer patterns. The results show improved concurrency handling, fewer MVCC-related failures, and higher transaction success rates.

3 Methodology

This study follows a design-and-evaluation methodology. First, a Hyperledger Fabric-based cardamom supply chain platform was designed to support traceability, pesticide testing, auction-based price discovery, smart contract-driven payments, and packet-level consumer verification. Second, the system was implemented using JavaScript chaincode, a React.js frontend, a Node.js REST API layer, IPFS-based off-chain storage, and a multi-organization Fabric network. Finally, the system was validated functionally and evaluated using Hyperledger Caliper under different transaction loads.

3.1 System Overview

The proposed system is a permissioned blockchain-based platform for the cardamom supply chain. It connects five stakeholder groups: farmers, aggregators, retailers, consumers, and the bank. Farmers submit cardamom lots with production details, and aggregators test and grade the lots for pesticide compliance and quality. Approved lots are offered to retailers through a bidding process. After purchase, retailers pack the lots into traceable packets, while consumers verify the packet history using the packet code. The bank creates and funds internal wallets to support smart contract-driven payments. All key events are recorded on the ledger to ensure transparency, accountability, and end-to-end traceability.

3.2 Blockchain Network Architecture

The blockchain network was implemented using Hyperledger Fabric with five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP. Each organization was configured with its own peer, MSP, and certificate authority. The organizations interact through a common channel, while CouchDB stores the world state. The network was created by extending the Fabric test network using customized Docker Compose, crypto-configuration, and channel configuration files. Raft was used as the ordering mechanism.

3.3 Smart Contract Workflow

The business logic was implemented as JavaScript chaincode. The bank creates and funds wallets using `createWallet()` and `depositMoney()`. Farmers submit cardamom lots using `submitProduce()`, and the testing fee is transferred to the selected aggregator. Aggregators record pesticide test results and grading details using `testCardamom()`. Approved lots are made available for retailer offers through `makeOffer()`, and farmers accept the highest valid offer using `acceptOffer()`. Retailers then pack accepted lots into packets using `packLotIntoPackets()`, while consumers purchase packets using `purchasePacket()` and verify provenance through lifecycle query functions. The overall sequence of stakeholder actions and smart contract functions is shown in Figure 3.

3.3.1 Traceability, Auction, and Incentive Mechanism. The traceability mechanism links each cardamom packet to its original lot, test result, grading details, and packing record. Testing and packing videos are stored off-chain in IPFS, while their hashes are stored on-chain for tamper-proof verification. This allows consumers to verify the product history using the packet code.

The auction mechanism supports price discovery for approved lots. Retailer offers are stored as separate composite-key records instead of being written into the lot record. This reduces shared-key updates and helps minimize MVCC conflicts during concurrent bidding. After the farmer accepts the highest valid offer, the smart contract transfers payment from the retailer wallet to the farmer wallet and updates lot ownership.

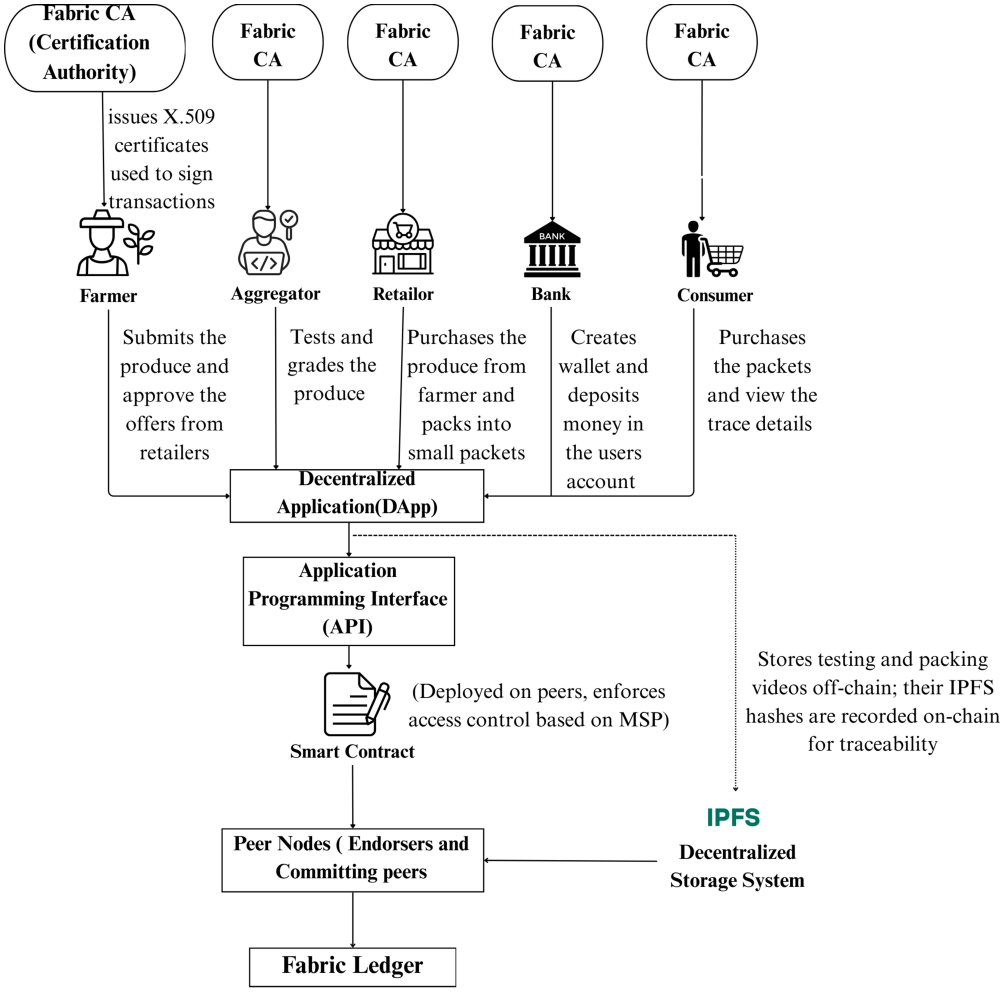

Fig. 1. System overview of the proposed cardamom supply chain platform

The incentive mechanism links pesticide compliance with farmer reputation. Approved lots improve the farmer rating and increase visibility to retailers, while rejected lots reduce the rating. This encourages farmers to follow safer cultivation practices and improves the credibility of quality-compliant producers. The integration of traceability, auction-based price discovery, and pesticide-compliance-based incentives is illustrated in Figure 4.

3.4 Application Layer and IPFS Integration

A React.js web application was developed with separate dashboards for farmers, aggregators, retailers, consumers, and the bank. The frontend communicates with a Node.js backend through REST APIs. The backend invokes chaincode functions using the Fabric Gateway SDK and communicates with Fabric peers through gRPC. Large files, such as testing and packing videos, are stored off-chain in IPFS. The generated IPFS content identifier is stored on-chain with the

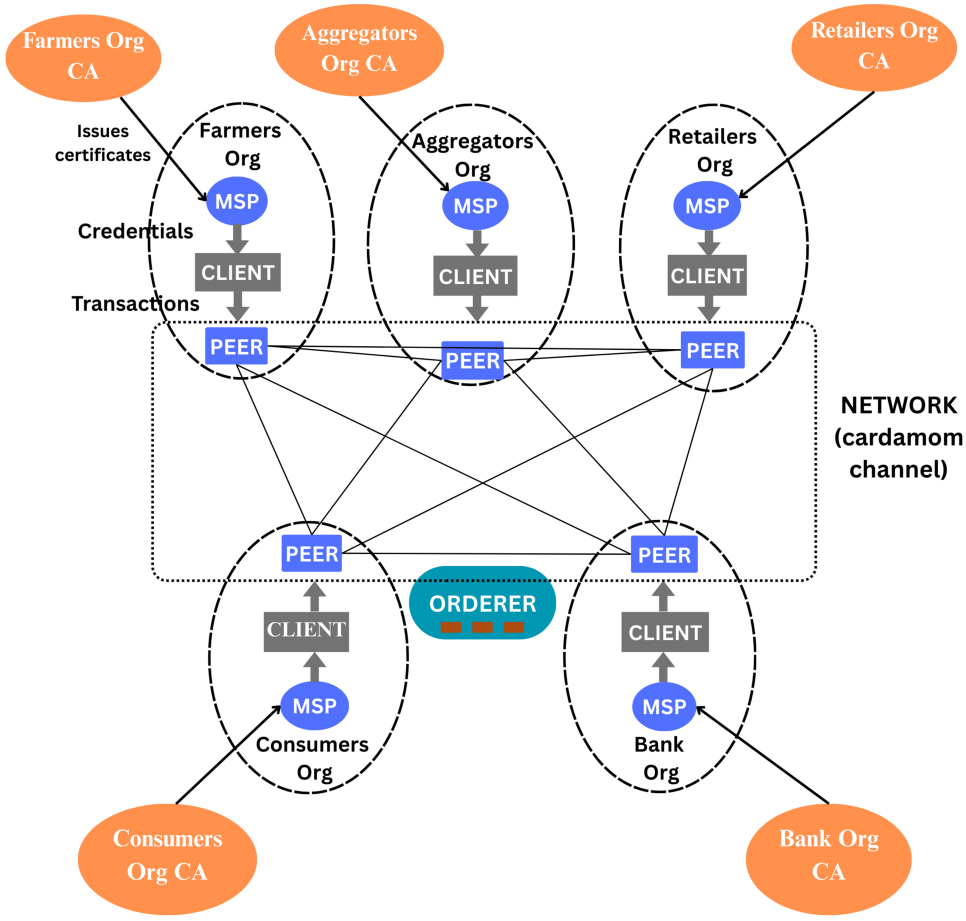

Fig. 2. Network Architecture

corresponding lot or packet record, allowing users to verify external evidence without increasing blockchain storage overhead.

3.5 Concurrency-Aware Design

The smart contract was designed to reduce MVCC conflicts during concurrent transactions. Wallet updates are recorded as append-only debit and credit entries using composite keys instead of repeatedly overwriting a shared balance key. Wallet balances are derived from these entries and periodically consolidated to reduce query overhead. Retailer offers are also stored as separate offer assets, preventing multiple retailers from updating the same lot record during bidding. The lot state is updated only when the farmer accepts the final offer, reducing shared-key contention.

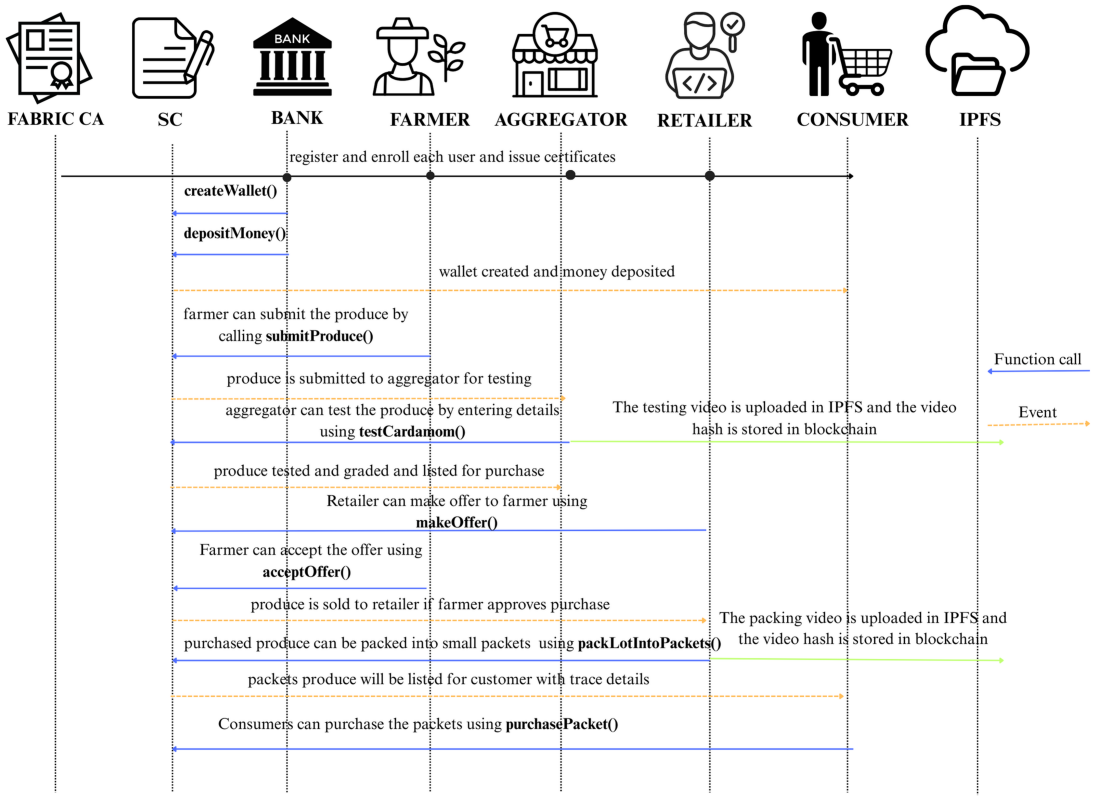

Fig. 3. Sequence of stakeholder actions and smart contract functions in the cardamom supply chain

4 System Testing and Validation

Following implementation, functional validation was performed to verify smart contract correctness, role-based access control, ledger consistency, wallet operations, IPFS hash recording, and traceability workflows.

4.1 Test Environment Setup

The smart contract was deployed on a local Hyperledger Fabric v2.5 test network using Docker containers. The network consisted of five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP, with one ordering service and corresponding certificate authorities. The JavaScript chaincode was tested through CLI commands, REST API calls from the Node.js backend, and the React-based frontend. IPFS was used to store testing and packing videos off-chain, while the generated hashes were recorded on-chain for verification.

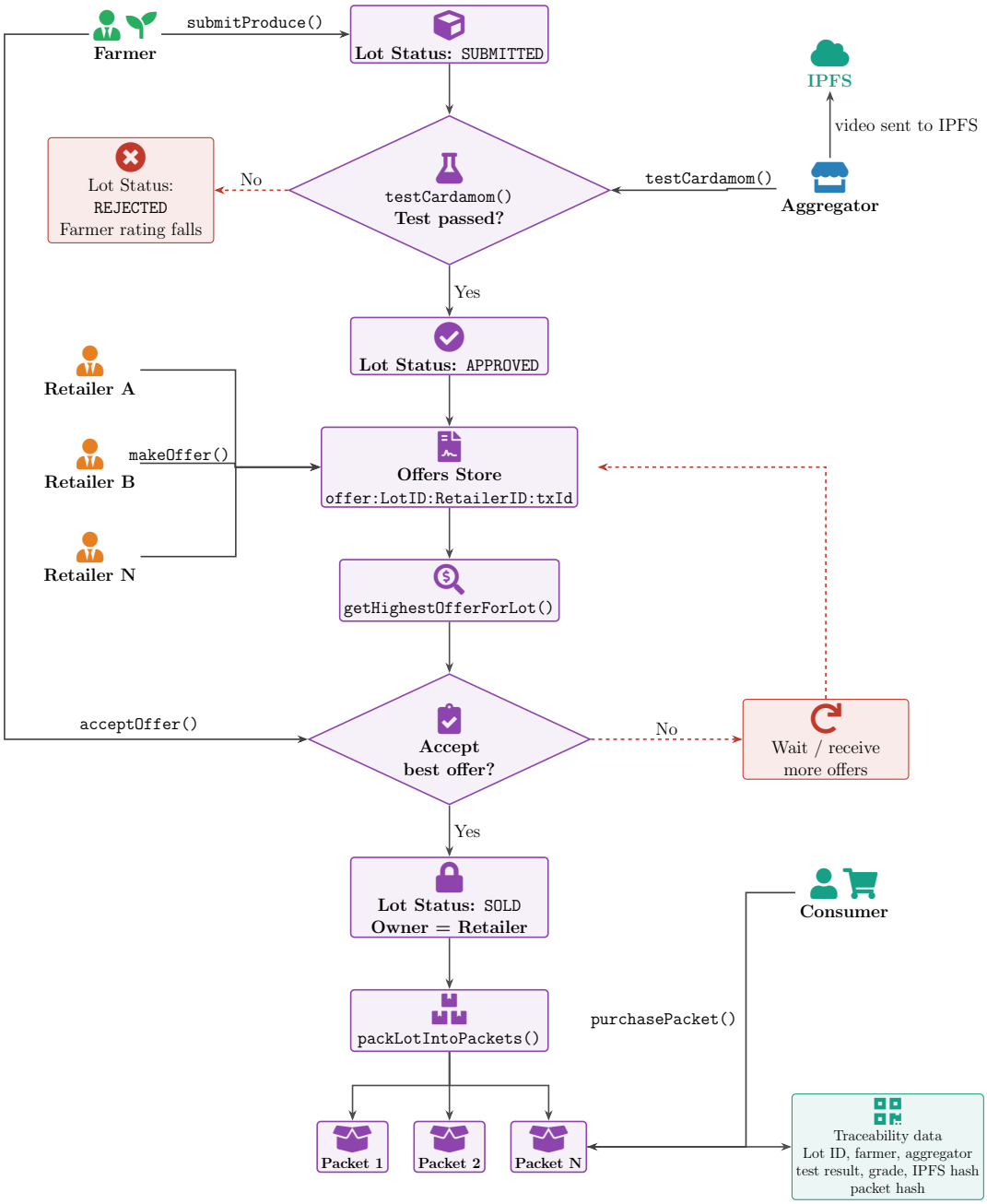

Fig. 4. Traceability, auction, and pesticide-compliance-based incentive mechanism

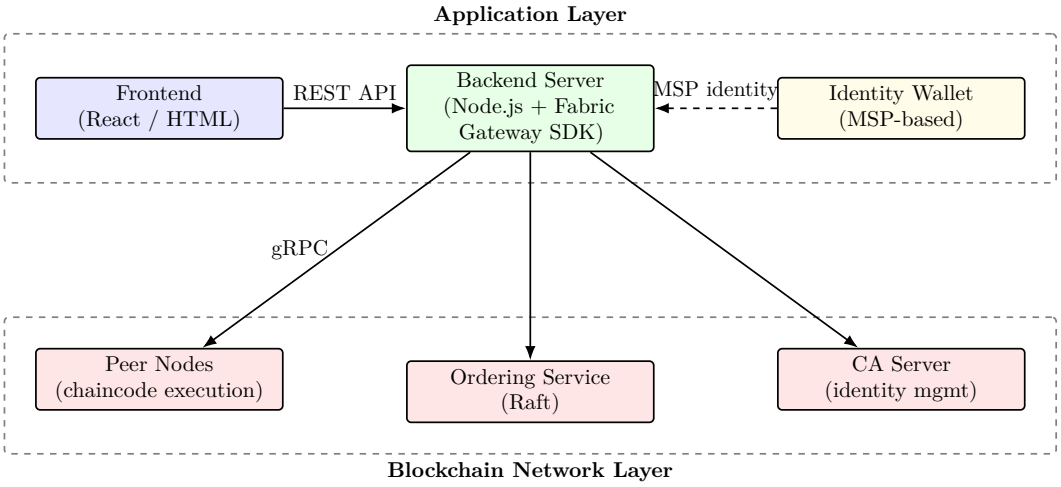

Fig. 5. Interaction between Frontend, Fabric SDK, and the Hyperledger Fabric Network

5 Performance Benchmarking with Hyperledger Caliper

Performance was evaluated using Hyperledger Caliper with the Fabric Gateway connector. The aim was to assess transaction reliability, write-path performance, read-path performance, scalability, and runtime behaviour of the proposed Hyperledger Fabric-based cardamom supply chain system. Caliper was used because it supports configurable workloads, concurrent client emulation, and automatic reporting of throughput, latency, send rate, and transaction success or failure counts.

5.1 Experimental Environment

The experiments were conducted on a workstation with a 12th Gen Intel Core i7-12700 processor, 32 GB RAM, Ubuntu 24.04.3 LTS on WSL2, Docker Desktop 4.49.0, and Docker Engine 28.5.1. The Fabric network was deployed using Docker containers and consisted of five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP. Each organization had one peer node, and the network used a single Raft-based ordering service and one application channel. TLS was enabled across the network.

All experiments were performed in a controlled single-host Docker environment to ensure repeatability. Therefore, the reported results should be interpreted as controlled laboratory measurements of relative function behaviour, concurrency effects, and resource pressure rather than production-scale throughput. The setup does not fully capture wide-area network latency, geographically distributed endorsement delay, or multi-host consensus overhead.

5.2 Workload and Transaction Load Design

Custom Caliper workload modules were developed for the main smart contract functions. Write workloads included *submitProduce()*, *testCardamom()*, *makeOffer()*, *acceptOffer()*, *packLotIntoPackets()*, and *purchasePacket()*. Read workloads included *getAllProduce()*, *getAllPackets()*, and provenance-related lookup functions such as *getPacketLifecycle()*.

Benchmark rounds were executed at increasing configured send rates of 50, 100, 200, 300, 400, and 500 TPS. Each round submitted 5000 transactions using five concurrent workers under a fixed-rate workload model. Each function was benchmarked independently to measure throughput, latency, send-rate behaviour, and failure patterns without interference from unrelated workloads.

5.3 Retrieval and Dataset Scaling

Retrieval performance was evaluated separately because read operations are affected by ledger growth, state-database access, and composite-key iteration. The benchmark environment was preloaded with increasing volumes of lot and packet records to analyse the effect of dataset size on query latency and throughput.

Two types of retrieval operations were considered. Operational queries such as *getAllProduce()* and *getAllPackets()* accessed the current world state stored in CouchDB. Provenance queries such as *getPacketLifecycle()* used transaction-history APIs to reconstruct the lifecycle of a packet from ledger history. These two query types were benchmarked separately because they follow different data-access patterns.

5.4 Network Latency and Resource Monitoring

To examine the effect of network conditions, experiments were conducted under both baseline and network-impaired settings. Artificial latency, jitter, and packet loss were introduced at the Docker container level to simulate communication disturbances between network components.

Runtime resource utilization was monitored during each benchmark round. System-level CPU usage was recorded using the Linux `top` command in batch mode, while container-level CPU, memory, and network I/O were collected using `docker stats`. These measurements were used to relate throughput and latency changes to resource pressure on peer, orderer, CouchDB, and client containers.

5.5 Performance Metrics

For each benchmark round, the recorded metrics included successful transactions, failed transactions, configured send rate, achieved send rate, throughput, minimum latency, maximum latency, average latency, and 95th percentile latency. Throughput was calculated as the number of successfully completed transactions or queries per second. Write latency was measured from transaction submission to confirmation, while read latency was measured from query submission to response receipt.

$$\text{Throughput}_{write} = \frac{\text{Number of successfully committed write transactions}}{\text{Total benchmark duration (s)}} \tag{1}$$

$$\text{Throughput}_{read} = \frac{\text{Number of successfully completed read transactions}}{\text{Total benchmark duration (s)}} \tag{2}$$

$$\text{Latency}_{write} = \text{Transaction confirmation time} - \text{Transaction submission time} \tag{3}$$

$$\text{Latency}_{read} = \text{Query response time} - \text{Query submission time} \tag{4}$$

$$\text{Send Rate} = \frac{\text{Successful transactions} + \text{Failed transactions}}{\text{Last submission time} - \text{First submission time}} \tag{5}$$

5.6 Performance Results

5.7 Function-wise Write Performance

Each core write function was evaluated independently under increasing send rates from 50 to 500 TPS using Hyperledger Caliper. The tested functions were *submitProduce()*, *testCardamom()*, *makeOffer()*, *acceptOffer()*, *packLotIntoPackets()*, and *purchasePacket()*. The objective was to compare throughput, latency, and failure behaviour across functions with different ledger-write complexity.

Table 4. Performance Results for `submitProduce` under Varying Load

Function	TPS	Success	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
`submitProduce`	50	5000	0	50.0	1.33	0.00	0.11	50.0
`submitProduce`	100	5000	0	100.0	1.31	0.00	0.09	99.9
`submitProduce`	200	5000	0	200.0	1.11	0.00	0.07	199.6
`submitProduce`	500	4994	6	495.2	6.91	0.10	4.45	294.8

As shown in Table 4, *submitProduce()* maintained stable performance at lower loads (up to 200 TPS), with latency increasing at 500 TPS. For the remaining write functions, performance at the maximum tested load of 500 TPS is summarized in Table 5.

Table 5. Performance Comparison of Remaining Smart Contract Write Functions at 500 TPS Load

Function	TPS	Success	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
`testCardamom`	500	4992	8	422.9	6.11	0.11	4.27	295.2
`makeOffer`	500	5005	5	499.4	0.26	0.01	0.09	416.3
`acceptOffer`	500	4996	4	497.8	9.21	0.09	6.48	261.4
`packLotIntoPackets`	500	4991	9	473.7	32.84	0.60	22.56	140.1
`purchasePacket`	500	4078	922	496.7	6.66	0.09	4.21	300.2

At 500 TPS, *makeOffer()* achieved the best scalability among the tested write functions, maintaining very low average latency and reaching a maximum throughput of 416.3 TPS. This result reflects its lightweight design and reduced shared-state updates.

In contrast, more contention-prone operations exhibited higher latency and failure rates. The *packLotIntoPackets()* function reached a throughput ceiling of about 140 TPS because each transaction creates multiple packet records, increasing endorsement and commit overhead. The *purchasePacket()* function achieved high throughput but produced higher failure counts because multiple clients attempted to purchase the same packet concurrently, causing MVCC conflicts on the shared packet asset.

5.8 Phase 2: Retrieval Function Benchmarking

The second phase evaluated read-only functions, including *getAllProduce()*, *getAllPackets()*, and *getPacketLifecycle()*. These functions do not modify ledger state but retrieve data using different access patterns. The *getAllProduce()* and *getAllPackets()* functions access the world state through composite-key scans and filtering, while *getPacketLifecycle()* reconstructs provenance using transaction-history traversal.

Table 6 shows that *getPacketLifecycle()* remained stable from 50 to 500 TPS, with no failures and very low average latency. Throughput closely followed the configured send rate, reaching 495.8 TPS at 500 TPS. This indicates that provenance queries can scale efficiently when history traversal is limited and no world-state updates are involved.

Table 6. Performance Results for `getPacketLifecycle` (Payload: 2908 bytes) under Varying Load

Function	TPS	Total Tx	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
getPacketLifecycle	50	5000	0	50.0	0.06	0.00	0.01	50.0
getPacketLifecycle	100	4995	0	99.9	0.02	0.00	0.01	99.9
getPacketLifecycle	200	5000	0	199.8	0.50	0.01	0.01	199.8
getPacketLifecycle	300	5000	0	299.4	0.03	0.01	0.01	299.3
getPacketLifecycle	400	5000	0	377.8	1.19	0.00	0.01	373.6
getPacketLifecycle	500	5000	0	496.2	0.10	0.00	0.02	495.8

The retrieval-function results show that world-state retrieval performance is strongly affected by payload size. Smaller payloads remained stable at lower and moderate loads, while larger payloads caused sharp latency growth, reduced throughput efficiency, and higher failure rates. This behaviour is mainly due to composite-key scanning, filtering, data serialization, and network transfer overhead. The results show that payload size is a major bottleneck for large read queries in Fabric-based supply chain applications. Table 8 presents the performance of the *getProduce()* function under varying payload sizes. The results demonstrate a strong dependency of query performance on the volume of data retrieved.

Table 7. Performance Results for `getAllPackets` under Varying Payload Sizes and Load

TPS	Success	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
Payload: 66 KB							
50	5000	0	50.0	0.06	0.00	0.01	50.0
100	5000	0	100.0	0.10	0.00	0.01	99.9
200	5000	0	199.8	0.07	0.00	0.01	199.8
300	5000	0	299.7	0.50	0.01	0.17	296.1
400	5000	0	397.9	7.70	0.04	3.94	285.6
500	4536	464	495.0	9.65	0.06	5.29	314.5
Payload: 330 KB							
50	5000	0	50.0	0.58	0.00	0.05	50.0
100	5000	0	100.0	19.27	0.11	10.81	77.4
200	3690	1310	199.5	35.25	0.12	21.81	99.7
300	3022	1978	299.6	34.36	0.22	23.37	118.4
400	2721	2279	397.7	33.66	0.49	24.59	130.4
500	2632	2368	497.9	36.23	0.47	27.31	125.8
Payload: 660 KB							
50	5000	0	50.0	26.55	0.15	13.33	41.8
100	1049	3941	99.8	45.19	0.00	17.10	74.5
200	386	4614	200.1	38.95	10.57	25.43	99.9
300	262	4733	299.2	29.89	11.41	20.21	110.2
400	0	5000	398.1	–	–	–	148.6
500	0	5000	499.5	–	–	–	148.2

For smaller payload sizes (94.3 KB), the function maintains stable performance up to 200 TPS with negligible latency and no failures. However, at 300 TPS, a sharp increase in latency is observed, with average latency rising to over 6 seconds, indicating the onset of system saturation. At higher loads (400–500 TPS), transaction failures begin to appear, and throughput deviates from the configured send rate. This behavior is primarily caused by increasing queuing delays at the peer level, where incoming requests exceed the processing capacity of endorsement and query execution.

As the payload size increases to 471.5 KB, performance degrades significantly even at moderate load levels. Latency increases sharply, exceeding 30 seconds at 100 TPS, and failure rates become substantial from this point onward. Throughput drops considerably, reflecting the combined impact of expensive world-state traversal, large data serialization, and increased network transfer overhead. The larger response size prolongs request processing time, leading to accumulation of concurrent requests in the system.

For the largest payload size (943 KB), the system exhibits severe performance limitations. Even at low load levels (50 TPS), failure rates are high and latency exceeds 20 seconds. Beyond 200 TPS, all transactions fail, indicating that the system is unable to handle large-volume data retrieval under concurrent workloads. This failure behavior is further exacerbated by endorser concurrency saturation, where the number of in-flight requests exceeds the peer's concurrency limit, resulting in request rejection (e.g., "exceeding concurrency limit" errors).

This behaviour is primarily attributed to the increasing cost of world-state traversal, data aggregation, and response serialization as payload size grows. Since *getProduce()* relies on composite key scans and filtering operations over the world state, larger result sets significantly increase computational and I/O overhead. These results highlight that payload size is a dominant factor influencing read performance and that large-scale data retrieval can become a critical bottleneck in blockchain-based applications.

5.9 Impact of Network Latency

To examine the effect of network conditions, additional experiments were conducted by introducing controlled impairments between Docker containers: 50 ms fixed latency, 20 ms jitter, and 1% packet loss. Since the impact of network variability is more visible at higher loads, Table 9 reports the results at 500 TPS.

Table 9 shows that network impairment substantially increases latency and reduces throughput, especially for complex write operations and large retrieval queries. Functions such as *submitProduce()*, *testCardamom()*, and *acceptOffer()* show higher average latency because endorsement, ordering, and commit phases are all affected by communication delay. The *packLotIntoPackets()* function is affected most among write operations due to write amplification from multiple packet records.

The *makeOffer()* function remains comparatively stable, with lower latency and higher throughput, because it performs lightweight ledger updates. In contrast, *purchasePacket()* shows higher failures because network delay increases the time window for concurrent access to the same packet asset, thereby increasing MVCC conflicts.

Retrieval functions show the strongest degradation under impaired network conditions. Both *getProduce()* and *getAllPackets()* involve larger response payloads, making them more sensitive to latency, jitter, packet loss, and peer-side request accumulation. The results indicate that read-path scalability is constrained by both payload size and request concurrency. Overall, the experiment confirms that network delay has a stronger impact on data-intensive and write-heavy functions than on lightweight transaction functions.

Table 8. Performance Results for getProduce under Varying Payload Sizes and Load

TPS	Total Tx	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
Payload: 94.3 KB							
50	5000	0	50.0	0.10	0.00	0.01	50.0
100	4996	0	99.9	0.04	0.00	0.01	99.8
200	5000	0	199.8	0.27	0.00	0.06	199.7
300	5000	0	299.1	14.07	0.02	6.84	191.5
400	4265	735	399.4	13.50	0.10	7.31	243.0
500	3518	1482	497.7	15.42	0.10	9.13	255.2
Payload: 471.5 KB							
50	5000	0	50.0	1.28	0.00	0.41	49.9
100	4215	785	100.1	53.97	0.18	32.32	55.2
200	3085	1915	199.4	54.74	0.17	35.90	73.1
300	2773	2227	282.5	55.16	0.23	37.70	80.4
400	2609	2391	399.4	58.87	0.56	42.14	79.2
500	2543	2457	498.5	65.63	0.54	48.86	73.2
Payload: 943 KB							
50	1548	3452	50.0	58.88	0.27	21.66	47.9
100	427	4573	100.0	36.51	0.32	16.37	93.3
200	0	5000	200.1	–	–	–	142.3
300	0	5000	299.1	–	–	–	136.5
400	0	5000	398.8	–	–	–	137.5
500	0	5000	496.2	–	–	–	138.0

Table 9. Performance Comparison of Smart Contract Functions at 500 TPS under Network Impairment

Function	TPS	Success	Failed	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
submitProduce	500	4996	4	482.8	15.99	0.93	12.38	191.8
testCardamom	500	4990	3	482.4	15.70	0.67	11.02	193.4
makeOffer	500	5880	8	443.8	5.63	0.00	1.79	368.5
acceptOffer	500	4994	6	472.0	14.03	1.38	12.07	212.8
packLotIntoPackets	500	4993	7	590.0	70.15	19.83	62.74	70.51
purchasePacket	500	3346	1654	475.4	12.56	1.05	10.80	220.6
getProduce (13.2 KB)	500	804	4196	476.0	294.15	2.12	186.46	16.6
getAllPackets (66 KB)	500	1385	3613	483.5	293.78	1.05	223.76	16.5

5.10 Runtime Resource Utilization

Table 10 summarizes CPU, memory, and network I/O usage for representative write and retrieval functions. Write-intensive functions generally show increased CPU utilization at higher loads, especially *packLotIntoPackets()* and *acceptOffer()*, because they involve multiple state updates and ledger writes. In contrast, *purchasePacket()* maintains relatively low CPU and memory usage, although its failures are mainly related to MVCC conflicts rather than resource exhaustion.

Retrieval functions show moderate CPU usage but higher network I/O, particularly for *getAllPackets()* and *getProduce()*, because they return larger payloads from the ledger. Memory usage remains stable across all functions, indicating that no abnormal memory growth or resource exhaustion occurred during benchmarking. Overall, the results suggest that the observed throughput and latency variations are mainly caused by transaction complexity, payload size, and concurrency effects rather than insufficient system resources.

Table 10. Runtime Resource Utilization of Smart Contract and Retrieval Functions at 500 TPS Load

Function	TPS	CPU Avg (%)	CPU Peak (%)	Mem Avg (MB)	Net I/O (MB)
submitProduce	500	26.58	82.8	369.37	7759.74
testCardamom	500	27.44	78.5	208.56	5865.42
acceptOffer	500	28.86	81.7	371.74	8400.26
makeOffer	500	25.54	86.0	297.45	2681.62
packLotIntoPackets	500	38.19	85.0	379.52	7017.50
purchasePacket	500	9.96	79.3	243.63	2577.23
getAllPackets	500	21.58	69.2	340.15	6974.11
getProduce	500	12.04	51.1	305.59	7657.60
getPacketDetails	500	8.43	49.3	231.02	2586.68

5.11 Optimization to Reduce MVCC Conflicts

MVCC conflicts are a major source of transaction failure in Hyperledger Fabric when concurrent transactions update the same ledger key. In the unoptimized design, wallet transfers directly overwrote shared wallet-balance keys, while offer handling modified the same lot record during *makeOffer()*. These read–modify–write patterns created shared-key contention under concurrent workloads.

To reduce these conflicts, the chaincode was redesigned using append-only records and composite keys. Wallet transfers were recorded as separate `walletTx` debit and credit entries instead of overwriting wallet balances. Retailer offers were also stored as independent `offer` assets using composite keys rather than being embedded in the lot record. This is achieved by moving the `getWalletBalance()` check outside the transfer function and the `getHighestOffer()` check outside `makeOffer()`. These validations need to be checked in the backend rather than in the chaincode. As a result, concurrent transactions generated unique ledger entries instead of repeatedly modifying shared state.

Table 11 compares the unoptimized and optimized chaincode at 200 TPS. The optimized design substantially reduced transaction failures. For example, *acceptOffer()* improved from 3963 failures to zero, while *submitProduce()* improved from 3827 failures to zero. The *makeOffer()* function also eliminated failures after separating offers from the lot record. Although *purchasePacket()* still recorded 866 failures, this was mainly due to application-level contention where multiple clients attempted to purchase the same packet. Overall, the results confirm that append-only composite-key records significantly improve concurrency stability in the proposed Fabric system.

Table 11. Performance Comparison of Unoptimized and Optimized Chaincode at 200 TPS

Function	TPS	Success	Fail	Send Rate	Max Lat (s)	Min Lat (s)	Avg Lat (s)	Throughput
Unoptimized Chaincode								
acceptOffer	200	1037	3963	200.1	2.20	0.08	0.99	199.8
makeOffer	200	4630	380	200.4	2.05	0.00	0.05	185.5
packLotIntoPackets	200	5000	0	200.0	13.91	0.25	12.79	109.1
purchasePacket	200	1235	3765	199.4	2.84	0.05	0.10	179.4
submitProduce	200	1173	3827	199.7	0.91	0	0.10	199.2
testCardamom	200	5000	0	199.9	0.73	0.04	0.10	199.5
Optimized Chaincode								
acceptOffer	200	5000	0	199.2	1.49	0.00	0.12	198.7
makeOffer	200	5010	0	199.6	2.04	0.00	0.05	185.0
packLotIntoPackets	200	5000	0	199.5	17.17	0.19	10.73	118.9
purchasePacket	200	4134	866	200.0	2.05	0.04	0.10	185.2
submitProduce	200	5000	0	200.0	1.11	0.00	0.07	199.6
testCardamom	200	5000	0	199.8	0.15	0.00	0.08	199.4

6 Results and Discussion

6.1 Functional Validation

Functional validation was performed using both manual and automated testing. Manual testing was conducted through the web interface, REST API calls, and CLI commands to verify realistic stakeholder interactions. Automated scripts were then used to execute repeated transaction flows and confirm stable ledger updates.

The validation covered wallet creation and funding, produce submission, pesticide testing and grading, offer creation and acceptance, packet generation, consumer purchase, and provenance queries. Each transaction was followed by query operations to confirm correct state transitions, ownership updates, wallet changes, lot and packet status updates, and metadata consistency. Role-based access was tested using MSP identifiers to ensure that restricted functions could be invoked only by authorized stakeholders.

IPFS integration was validated by checking whether testing and packing video hashes were correctly stored on-chain and matched the corresponding off-chain files. These tests confirmed that the system preserved traceability and data consistency across the complete cardamom supply chain workflow.

6.2 Smart Contract Execution Summary

Table 12 summarizes the main smart contract functions and their validated outcomes. The validation confirms that the implemented chaincode correctly supports wallet operations, produce submission, quality testing, auction-based offer handling, packetization, consumer purchase, and traceability verification.

Overall, the validation confirmed that the system executed the intended stakeholder workflows correctly and maintained consistent ledger, wallet, and traceability records.

6.3 Practical Implications and Limitations

The system is suitable for practical cardamom supply chain scenarios where transactions are usually asynchronous and occur at lower rates than the stress-test conditions. Under realistic operating loads, the system can support traceability, quality verification, auction-based price discovery, and smart contract-driven payments.

Table 12. Smart Contract Execution Validation

Function	Validated Outcome
`createWallet()`, `depositMoney()`	Wallets were successfully created and token balances initialized for all stakeholders.
`submitProduce()`	Farmers submitted produce lots; testing fees were correctly deducted and transferred to aggregators.
`testCardamom()`	Test results and grading data were recorded; lot status updated based on quality evaluation.
`makeOffer()`, `acceptOffer()`	Retailers submitted offers; farmers accepted valid offers and payments were transferred accordingly.
`packLotIntoPackets()`	Retailers generated traceable packets (100gâ€“1kg) with associated metadata stored on-chain.
`purchasePacket()`	Consumers purchased packets and corresponding financial transactions were successfully executed.

However, the reported results should be interpreted as controlled laboratory measurements rather than production-scale throughput values. The experiments were conducted in a single-host Docker environment using WSL2, which does not fully capture wide-area network latency, geographically distributed endorsement, or multi-host ordering overhead. Future work should evaluate the system in a distributed deployment and optimize read-heavy functions through pagination, indexing, caching, and improved query design.

7 Conclusion

In this study, we proposed and evaluated a blockchain-based system designed to enhance traceability and financial integrity within the agri-food supply chain. The system leverages a permissioned blockchain network to immutably record critical lifecycle events, including produce submission, pesticide testing outcomes, grading results, and financial transactions. By establishing a tamper-proof record of events, the system ensures transparency and accountability among all stakeholders—farmers, aggregators, retailers, banks, and consumers.

Smart contracts are utilized to automate the enforcement of business rules, particularly for financial settlements. Payments are disbursed automatically based on predefined conditions such as passing pesticide tests or completing purchase agreements. This mechanism significantly reduces reliance on manual coordination, eliminates intermediaries, and minimizes delays in financial transactions. Moreover, the integration of on-chain pesticide compliance enforcement—via immutable test logs and traceable produce histories—acts as a regulatory check. It discourages the excessive use of harmful chemicals and promotes adherence to sustainable farming practices. Farmers are incentivized through faster payments, price premiums for pesticide-compliant lots, and access to broader export markets. In addition, positive compliance records can improve a farmer's rating on the blockchain, which in turn enhances trust with buyers and financial institutions, enabling easier access to credit and future contracts.

Furthermore, the system incorporates a blockchain-based auction mechanism in which retailers place ascending offers on approved lots. Each bid is immutably recorded on the ledger under a unique key, and the smart contract enforces that every new offer must exceed the previous highest bid. This ensures transparent and competitive price discovery, preventing exploitation in traditional opaque auction settings. By recording every bid immutably and validating bid increments on-chain, the mechanism discourages collusion, eliminates back-dating or bid withdrawal, and

guarantees fairness. Farmers benefit by reliably receiving the best available price for their produce, while retailers gain confidence in the authenticity and quality of lots. Together with the incentive framework, the auction design not only enhances financial transparency but also aligns market behavior with sustainable and equitable trading practices.

The system underwent extensive validation, including both functional testing and performance benchmarking using Hyperledger Caliper. The results demonstrated its robustness, scalability, and fault tolerance under realistic workloads. While most operations performed reliably, the system exhibited occasional transaction failuresâ€"particularly in the purchasePacket() function due to Multi-Version Concurrency Control (MVCC) conflicts under high concurrency. Additionally, some retrieval-heavy functions, such as getAllPackets() and getProduce(), showed increased latency as the ledger size grew, indicating potential bottlenecks in read performance that could impact system responsiveness over time. Nonetheless, traceability and transaction correctness were consistently maintained across all scenarios.

Looking ahead, future work will focus on integrating Internet of Things (IoT) technologies to further enhance the system's capabilities. Real-time sensor data—such as color, moisture content, size uniformity, and contaminant levels—will be captured at the point of origin using embedded devices. This data will be securely transmitted to the blockchain via cryptographic hashes and authenticated APIs, ensuring tamper-proof provenance. Such integration will facilitate objective, data-driven grading and pesticide residue assessments, enabling dynamic pricing models based on quantifiable quality attributes. In parallel, the system can be extended to support supply chain finance, where verified quality records and transaction histories are leveraged to unlock credit from banks or cooperatives, reducing farmers' dependence on informal lenders. Weather-indexed crop insurance can also be embedded, with smart contracts triggering automated payouts based on trusted meteorological feeds—such as rainfall or temperature deviations—to protect farmers against adverse weather events. In parallel, similar contract logic can use certified pesticide test results to settle claims related to produce rejection, thereby covering both climatic and compliance risks. Together, these mechanisms establish a robust cyber-physical infrastructure that not only supports real-time quality control but also strengthens financial resilience, enhances stakeholder trust, and improves the agility and scalability of agri-food supply chains.

Code Availability

The source code, configuration files, and benchmarking scripts used in this study are publicly available at:
https://github.com/dhanishjose123/fabric-traceability-cardamom

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used **DeepL** and **ChatGPT** in order to improve language and readability. After using these tools, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Author 1: Writing – original draft, Writing – review & editing, Validation, Methodology, Conceptualization.
Author 2: Writing – review & editing, Validation, Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by **Anonymous Institution**.